Classifieurs CNN et SmallBERT pour extraction de features

KIOSSOU Hermine - 22400380

Année universitaire 2025–2026

Introduction

Ce rapport présente la conception et l'évaluation de deux classifieurs complémentaires : un réseau de neurones convolutionnel (CNN) pour l'extraction de features visuelles et un modèle SmallBERT pour l'extraction de features textuelles. Ces deux architectures constituent des étapes intermédiaires essentielles pour la construction d'un modèle CLIP multimodal. L'objectif principal n'est pas d'optimiser les performances de classification, mais d'obtenir des encodeurs fonctionnels capables d'apprendre des représentations pertinentes dans leurs domaines respectifs.

1. Réseau de neurones convolutionnel (CNN)

1.1 Justification de l'architecture

Nous avons opté pour un réseau de neurones convolutionnel pour l'extraction de features visuelles en raison de sa capacité à capturer les hiérarchies spatiales dans les images. Les couches convolutionnelles permettent un apprentissage progressif de représentations de complexité croissante : les premières couches détectent des contours et textures simples, tandis que les couches profondes identifient des motifs complexes et des objets.

Ce classifieur s'inscrit comme une étape intermédiaire dans notre projet. Bien que nous l'entraînons à classifier les 4 catégories (ball, bike, dog, water), l'objectif final est d'extraire son encodeur pour le modèle CLIP. Conformément aux consignes du projet, nous ne cherchons pas à optimiser ce classifieur, mais plutôt à obtenir un modèle fonctionnel capable d'apprendre des représentations pertinentes.

1.2 Architecture détaillée

Notre architecture CNN se compose de deux blocs convolutionnels successifs avec une augmentation progressive du nombre de filtres (32 → 64), permettant au réseau d'apprendre des représentations visuelles de complexité croissante.

Premier bloc convolutionnel : Le premier bloc est constitué d'une couche Conv2D comprenant 32 filtres de taille 3×3, associée à une fonction d'activation ReLU et à un padding de type *same*. Cette couche est suivie d'un MaxPooling2D (2×2), qui réduit la résolution spatiale des images de 224×224 à 112×112. Un Dropout de 0.3 est ensuite appliqué afin d'introduire une première forme de régularisation et de limiter le surapprentissage. Ce bloc comporte 896 paramètres entraînables.

Deuxième bloc convolutionnel : Le second bloc reprend une structure similaire avec une couche Conv2D de 64 filtres (3×3), une activation ReLU et un padding *same*. Le MaxPooling2D (2×2) réduit cette fois les dimensions spatiales de 112×112 à 56×56. Un Dropout de 0.4 est ajouté afin de renforcer progressivement la régularisation. Ce bloc totalise 18 496 paramètres entraînables.

Aplatissement et couches denses : Après les blocs convolutionnels, nous utilisons une couche Flatten qui transforme les feature maps 3D (56×56×64) en un vecteur 1D de 200 704 dimensions. Ce vecteur de haute dimension capture l'ensemble des informations spatiales extraites par les couches convolutionnelles. Une couche Dense de 128 dimensions avec activation ReLU permet ensuite de créer une représentation compacte et discriminante des features visuelles. Cette compression dimensionnelle force le réseau à ne conserver que les informations les plus pertinentes pour la classification. Un Dropout de 0.5 est appliqué après cette couche pour renforcer la régularisation, particulièrement crucial avec un dataset limité de 600 images (480 train, 120 validation).

Couche de classification : La couche finale est un Dense de 4 neurones avec une activation softmax, produisant une distribution de probabilités sur les quatre classes cibles. Cette architecture compte au total 25,7 millions de paramètres entraînables.

1.3 Stratégie de régularisation progressive

La stratégie de Dropout adoptée est progressive (0.3 → 0.4 → 0.5) : nous appliquons une régularisation de plus en plus forte au fur et à mesure que le réseau devient profond. Cette approche permet de conserver une certaine flexibilité dans les premières couches (qui apprennent des features basiques) tout en imposant des contraintes plus fortes sur les couches supérieures (qui apprennent des représentations de haut niveau plus sujettes au surapprentissage).

1.4 Hyperparamètres et callbacks

Les hyperparamètres d'entraînement sont les suivants : epochs maximum de 30, batch size de 32 images par batch, optimizer Adam avec learning rate par défaut (5e-4), loss categorical_crossentropy adapté aux labels one-hot encodés, et un split des données en 80% train et 20% validation via ImageDataGenerator.

Pour optimiser l'entraînement et garantir la sauvegarde du meilleur modèle, nous avons implémenté deux callbacks complémentaires de Keras. Le callback EarlyStopping surveille la métrique val_loss et arrête automatiquement l'entraînement si celle-ci cesse de s'améliorer, avec une patience de 10 epochs. Le paramètre restore_best_weights=True garantit que les poids finaux du modèle correspondent à la meilleure epoch. Le callback ModelCheckpoint sauvegarde automatiquement le modèle dans le fichier cnn_minimal.keras à chaque fois que la val_loss s'améliore, avec save_best_only=True pour éviter d'écraser le fichier avec des versions moins performantes.

1.5 Résultats et analyse

L'entraînement s'est déroulé sur 27 epochs avant que l'EarlyStopping ne se déclenche, indiquant que la validation loss n'a pas progressé pendant 10 epochs consécutifs après l'epoch 17. Le ModelCheckpoint a automatiquement restauré les poids correspondant à cette meilleure epoch.

Métriques du meilleur modèle (Epoch 17) :

- Train accuracy : 0.577 (57.7%)
- Val accuracy : 0.375 (37.5%)
- Train loss : 0.990
- Val loss : 1.357

Un écart significatif de 20,2 points entre l'accuracy sur l'ensemble d'entraînement (57.7%) et celle sur l'ensemble de validation (37.5%) met en évidence un phénomène de surapprentissage. La validation loss (1.357) dépasse le train loss (0.990) de 37%, ce qui confirme les difficultés du modèle à généraliser sur de nouvelles données.

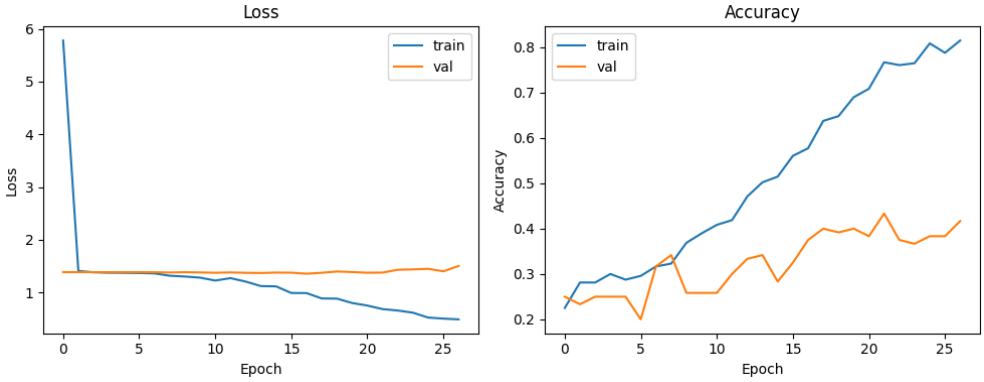

FIGURE 1 – Évolution de la loss et de l'accuracy durant l'entraînement du CNN

L'analyse des courbes d'apprentissage met en évidence des dynamiques distinctes entre les phases d'apprentissage et de validation, témoignant d'un comportement typique de surapprentissage. La courbe de train loss présente une décroissance rapide et régulière tout au long de l'entraînement, passant d'une valeur initiale élevée (environ 5.8) à des valeurs nettement plus faibles (aux alentours de 0.5) à l'epoch 27. En revanche, la validation loss se stabilise très tôt, oscillant autour de 1.38-1.45 dès les premières epochs et ne montre aucune amélioration significative au-delà de l'epoch 17.

La courbe d'accuracy d'entraînement progresse de manière quasi-monotone, passant d'environ 25% (performance aléatoire pour 4 classes) à plus de 82% en fin d'entraînement. L'accuracy de validation présente un comportement radicalement différent : elle oscille fortement entre 20% et 43% sans progression claire. Les fluctuations importantes suggèrent une instabilité dans les prédictions sur le set de validation.

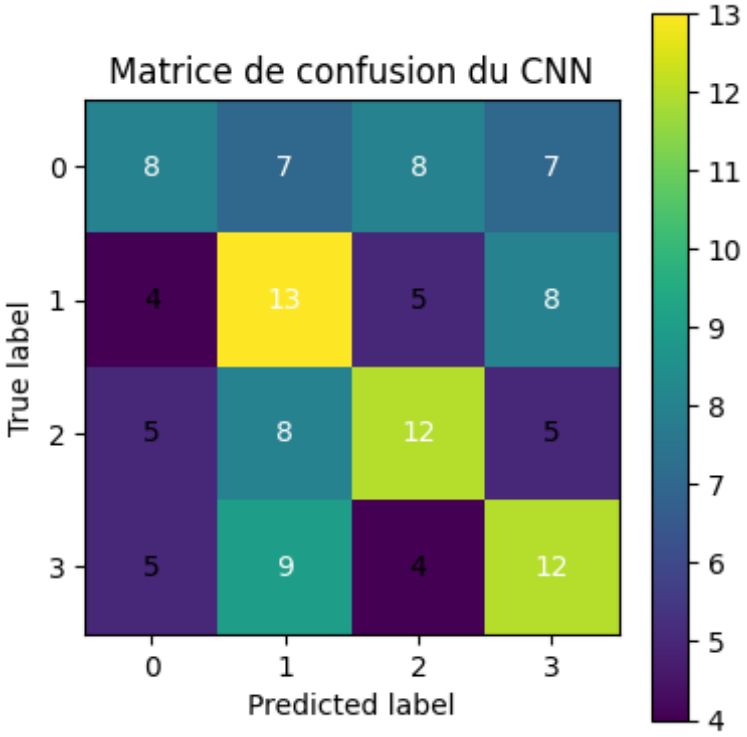

FIGURE 2 – Matrice de confusion du CNN sur l'ensemble de validation

1.6 Commentaires sur les résultats

Ces résultats modestes sont acceptables pour notre objectif. Nous ne cherchions pas à optimiser ce classifieur, mais simplement à obtenir un encodeur capable d'extraire des représentations visuelles pour CLIP. Plusieurs facteurs limitent les performances. Le dataset est restreint (120 images par classe), favorisant le surapprentissage. Les classes présentent aussi des ambiguïtés : chiens dans l'eau, vélos près de plans d'eau, ballons sur l'herbe, etc. Malgré une accuracy de validation modeste, le modèle a appris à extraire des features utiles (textures, formes, couleurs). C'est cet encodeur visuel – les couches convolutionnelles sans la couche de classification – que nous réutiliserons pour CLIP.

2. Modèle SmallBERT pour classification de texte

2.1 Objectif et justification

L'objectif de cette étape est de construire un classifieur de texte capable de prédire la classe associée à une image (ball, bike, dog, water) uniquement à partir de sa description textuelle. Ce classifieur constitue une étape intermédiaire essentielle avant la construction du modèle CLIP : il permet de vérifier que l'architecture

SmallBERT est capable de comprendre les descriptions textuelles et d'en extraire des représentations sémantiques pertinentes.

SmallBERT est une variante allégée de BERT, spécialement conçue pour les applications nécessitant un bon compromis entre performance et efficacité computationnelle. Contrairement au BERT original, SmallBERT réduit significativement la taille du modèle tout en conservant l'essentiel de sa capacité de compréhension sémantique. Cette architecture repose sur le mécanisme d'attention des Transformers, qui permet de capturer les relations contextuelles entre tous les mots d'une phrase simultanément, contrairement aux approches séquentielles.

2.2 Architecture détaillée

Notre classifieur de texte s'articule autour de quatre composantes principales : une couche d'entrée pour les séquences tokenisées, un encodeur SmallBERT qui extrait les représentations contextuelles, une couche de pooling qui agrège ces représentations, et enfin une tête de classification.

Couche d'entrée : La couche d'entrée reçoit des séquences d'entiers de longueur fixe 32, correspondant aux phrases préalablement prétraitées et vectorisées par la couche TextVectorization. Chaque entier représente l'identifiant d'un token appartenant au vocabulaire appris. Le paramètre dtype="int32" indique que les entrées sont des entiers, et non des embeddings pré-calculés.

Encodeur SmallBERT : Le cœur du modèle est l'encodeur SmallBERT, une architecture Transformer complète qui transforme les séquences de tokens en représentations contextuelles riches. Cette composante compte 1 944 832 paramètres entraînables, représentant 99.97% des paramètres totaux du modèle. La configuration de l'encodeur comprend : embed_dim=128 (dimension des embeddings de tokens), num_heads=4 (nombre de têtes d'attention dans le mécanisme Multi-Head Attention), ff_dim=256 (dimension des couches feed-forward internes), et num_layers=2 (nombre de blocs Transformer empilés).

L'encodeur SmallBERT applique séquentiellement : une couche d'embedding pour la conversion des IDs de tokens en vecteurs denses de dimension 128, un encodage positionnel pour l'ajout d'informations de position, et deux blocs Transformer comprenant chacun Multi-Head Self-Attention, Layer Normalization avec Residual Connection, Feed-Forward Network, et à nouveau Layer Normalization avec Residual Connection.

Couche de pooling (MaskedMean) : Contrairement à BERT standard qui utilise un token [CLS] dédié à la classification, SmallBERT ne possède pas ce token spécial. Nous devons donc agréger manuellement les représentations des 32 tokens pour obtenir un vecteur unique représentant la phrase entière. Pour cela, nous utilisons une couche personnalisée MaskedMean qui calcule la moyenne pondérée des embeddings de tokens en ignorant le padding (les tokens de valeur 0). Cette approche présente deux avantages majeurs : robustesse au padding (les phrases courtes complétées avec des zéros ne sont pas biaisées par des tokens artificiels sans

signification) et représentation équitable (tous les tokens significatifs contribuent également au vecteur final).

Tête de classification : Après le pooling, nous disposons d'une représentation compacte de chaque phrase. La classification finale s'effectue en deux étapes : un Dropout de 0.1 appliqué au vecteur poolé pour une régularisation légère, puis une couche dense finale avec 4 neurones (un par classe) et l'activation softmax qui transforme le vecteur de 128 dimensions en une distribution de probabilités sur les 4 classes.

2.3 Hyperparamètres et callbacks

Les hyperparamètres d'entraînement sont : epochs maximum de 30, batch size de 32 séquences par batch, optimizer Adam avec learning rate = 1e-4, loss sparse_categorical_crossentro et un split des données en 80% train et 20% validation. Le choix d'un learning rate relativement faible (1e-4) est crucial pour les Transformers : ces architectures sont sensibles aux variations brusques des gradients.

Nous avons implémenté la même stratégie de callbacks que pour le CNN : EarlyStopping avec une patience de 10 epochs surveillant la val_loss, et ModelCheckpoint sauvegardant automatiquement le modèle dans le fichier best_smallbert_model.keras à chaque amélioration de la val_loss.

2.4 Résultats et analyse

L'entraînement s'est déroulé sur 24 epochs avant que l'EarlyStopping ne se déclenche, indiquant que la validation loss n'a pas progressé pendant 10 epochs consécutives après l'epoch 14.

Métriques du meilleur modèle (Epoch 14) :

- Train accuracy : 0.985 (98.5%)
- Val accuracy : 0.742 (74.2%)
- Train loss : 0.088
- Val loss : 0.608

Avec 74.2% d'accuracy de validation, SmallBERT surpasse significativement le CNN (37.5%), mettant en évidence que les descriptions textuelles sont plus discriminantes que les images pour cette tâche de classification. Un écart substantiel de 24.3 points entre l'accuracy d'entraînement (98.5%) et celle de validation (74.2%) indique la présence de surapprentissage modéré. La validation loss (0.608) est 6.9 fois supérieure à la train loss (0.088), confirmant que le modèle mémorise partiellement les données d'entraînement.

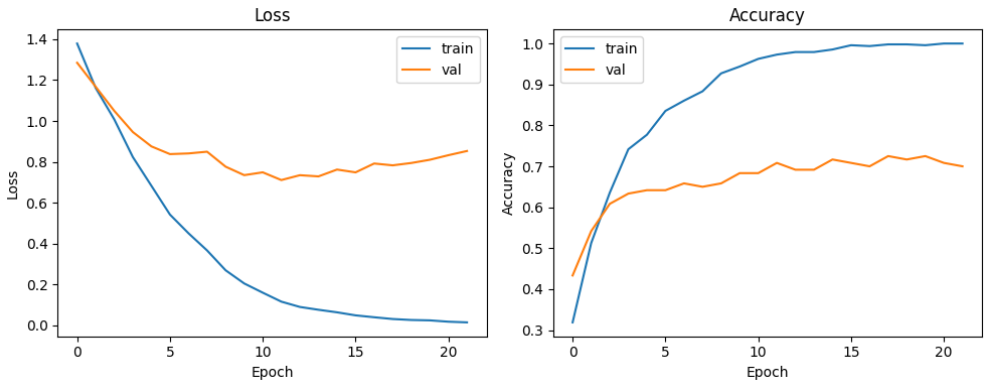

FIGURE 3 – Évolution de la loss et de l'accuracy durant l'entraînement de SmallBERT

L'analyse des courbes d'entraînement révèle une dynamique d'apprentissage distincte en deux phases. Durant la phase d'apprentissage rapide (Epochs 1-11), la train accuracy passe de 31% à 94.6% et la val accuracy de 39.2% à 75.8%. Durant la phase de surapprentissage (Epochs 12-24), la train accuracy continue vers 100% tandis que la val accuracy stagne entre 74-77%. La val_loss cesse de s'améliorer, oscillant autour de 0.61-0.74.

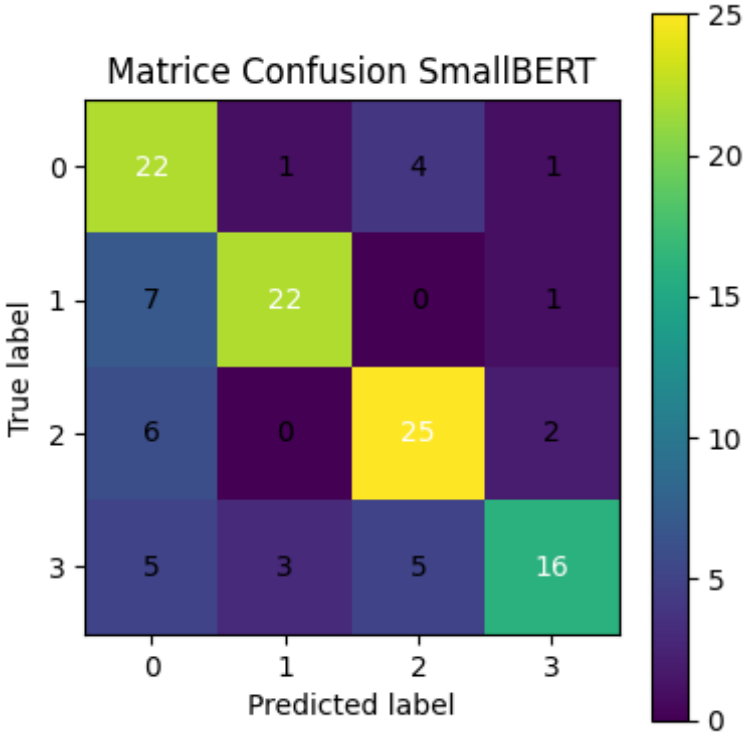

FIGURE 4 – Matrice de confusion de SmallBERT sur l'ensemble de validation

2.5 Analyse par classe

L'analyse détaillée révèle des performances hétérogènes entre les classes, avec des F1-scores variant entre 0.65 et 0.89. La classe Bike obtient la meilleure performance (F1=0.89, precision=1.00, recall=0.80) sans aucun faux positif. Le modèle identifie les vélos avec une certitude absolue. La classe Dog obtient une bonne performance (F1=0.74, precision=0.70, recall=0.79) avec un recall élevé mais une precision modérée. La classe Ball obtient un résultat satisfaisant (F1=0.71, precision=0.78, recall=0.64) avec un recall relativement faible, le modèle manquant environ 1 ballon sur 3. La classe Water est la plus difficile (F1=0.65, precision=0.58, recall=0.72) avec la precision la plus faible, le modèle générant de nombreux faux positifs.

2.6 Comparaison CNN vs SmallBERT

SmallBERT démontre une capacité d'apprentissage nettement supérieure à celle du CNN : il atteint 98.5% sur le train (contre 57.7% pour le CNN) et 74.2% sur la validation (contre 37.5% pour le CNN). Cette différence s'explique par la richesse sémantique des descriptions textuelles : les phrases comme "a dog running on the beach" contiennent des informations explicites et non ambiguës, contrairement aux images qui peuvent être visuellement complexes ou contenir plusieurs objets simultanément.

SmallBERT surpasse le CNN car les captions contiennent souvent le mot de la classe ("dog", "bike"), alors que l'image nécessite une interprétation visuelle complexe. Le mécanisme d'attention du Transformer peut se concentrer sur les mots clés de la phrase et ignorer les mots moins informatifs. De plus, le vocabulaire et les structures de phrases sont plus homogènes que la diversité visuelle des images.

Conclusion

Ce projet a permis de construire deux encodeurs complémentaires : un CNN pour les features visuelles et un SmallBERT pour les features textuelles. Bien que le CNN affiche des performances modestes (37.5% de validation accuracy), il a appris des représentations visuelles pertinentes qui seront utiles pour CLIP. SmallBERT, avec 74.2% de validation accuracy, démontre une capacité remarquable à extraire des représentations sémantiques des descriptions textuelles.

Ces deux encodeurs, une fois libérés de leurs couches de classification, fourniront les fondations pour construire un modèle CLIP capable d'apprendre des représentations multimodales alignées entre images et textes. L'objectif n'était pas d'optimiser les performances de classification, mais d'obtenir des encodeurs fonctionnels – objectif pleinement atteint.